

Paper Presentation: MegaBlocks: Efficient Sparse Training with Mixture-of-Experts

Eungseo Kim

Seoul National University, Biointelligence Lab

November 2, 2025

Original Paper Information

Original Authors

Trevor Gale¹, Deepak Narayanan², Cliff Young³, Matei Zaharia¹

Publication

MLSys 2023 - Proceedings of Machine Learning and Systems

Affiliations

¹ Stanford University, Stanford, California, USA

² Microsoft Research, Redmond, Washington, USA

³ Google Research, Mountain View, California, USA

Table of Contents

- 1 TL;DR
- 2 Introduction
- 3 Content
- 4 Discussion
- 5 References

The Problem

- Sparsity enables efficient scaling through conditional computation
- MoE leverages sparsity by routing tokens to specific experts
- Dynamic routing leads to uneven token distribution across experts
- Existing approach: Drop excess tokens, pad insufficient experts

MegaBlocks Solution

- Never drop tokens!
- Adapt block-sparse operations
- No performance loss, significant Expert Parallelism speedup

Key Concepts to be Covered

- **Sparsity:** A concept that activates only necessary components through conditional computation for computational efficiency
- **MoE (Mixture-of-Experts):** An architecture that combines multiple expert models to scale model capacity efficiently
- **Dynamic Routing:** A mechanism that dynamically assigns each token to appropriate experts
- **Expert Parallelism:** A technique that distributes multiple experts across different devices or processors for efficient parallel computation

What is Sparsity?

Conditional Computation

Sparsity uses the idea of **conditional computation**. While in dense models all parameters are used for all inputs, sparsity allows us to only run some parts of the whole system.

Scaling Without Computation Overhead

Shazeer's work showed that conditional computation allows scaling model size without increasing computation proportionally—thousands of experts can be used in each MoE layer.

Challenges

- **Uneven batch sizes:** If 10 tokens are batched, 5 might go to one expert and 5 to different experts
- **Underutilization:** This leads to uneven batch sizes and inefficient hardware usage

What is Mixture-of-Experts (MoE)?

Definition

MoE is an architecture that combines multiple expert models to achieve high capacity efficiently by routing each input to a small subset of experts. MoE leverages sparsity by routing tokens to specific experts, enabling conditional computation without activating all parameters.

Key Components

- **M experts:** neural networks specializing in different sub-problems
- **Router:** decides which experts to consult for each input
- **Gating network:** computes mixture weights for experts; typically linear

Mathematical Formulation

For each token x (or batch X), MoE routing works as follows
[Shazeer et al., 2017, Fedus et al., 2022]:

- Router computes logits:
 $z_m = x^T \theta^{(m)}$ for each expert
 $m \in [M]$, where $\theta^{(m)} \in \mathbb{R}^d$ is the router parameter for expert m
- Gate probabilities: $p = \text{softmax}(z)$
- Top-k selection: choose k experts with highest probabilities
- Expert networks $\{\phi_m\}_{m=1}^M$ (typically FFNs) process routed tokens
- Output: weighted combination of selected experts

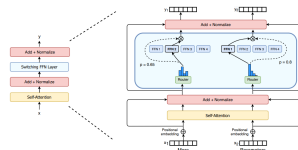


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = “More” and x_2 = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Figure 1: Switch Transformer architecture.

MoE Architecture in MegaBlocks

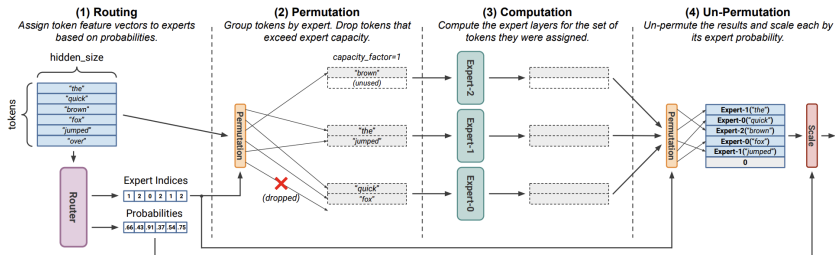


Figure 1. A Mixture-of-Experts Layer. Shown for $num_experts=3$, $top_k=1$ and $capacity_factor=1$ with the prevalent, token dropping formulation. **First (1)**, tokens are mapped to experts by the router. Along with expert assignments, the router produces probabilities that reflect the confidence of the assignments. **Second (2)**, the feature vectors are permuted to group tokens by expert assignment. If the number of tokens assigned to an expert exceeds its capacity, extra tokens are dropped. **Third (3)**, the expert layers are computed for the set of tokens they were assigned as well as any padding needed for unused capacity. **Lastly (4)**, the results of the expert computation are un-permuted and weighted by the router probabilities. The outputs for dropped tokens are shown here set to zero.

Figure 2: MoE architecture introduced by MegaBlocks.

So Why Dynamic?

What is Dynamic Routing?

Dynamic routing is a mechanism where the set of selected experts depends on the input. Different inputs are routed to different subsets of experts based on their characteristics.

Key Insight

When input x changes, the set of experts selected also changes:

- Different tokens x_i and x_j may be routed to completely different expert subsets
- This enables **input-adaptive computation**: experts specialize in different patterns
- Some experts may handle syntax, others semantics—depending on the token

Expert Parallelism

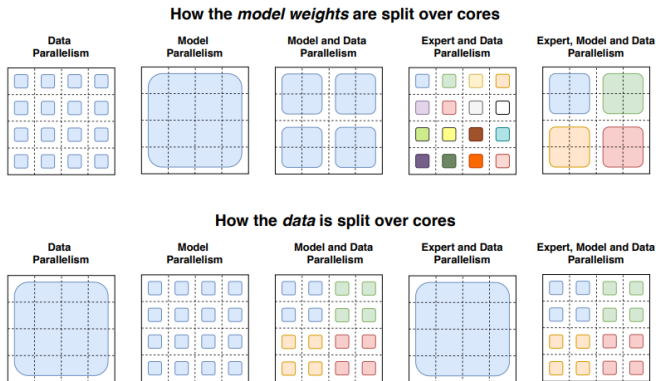


Figure: Expert parallelism architecture [Fedus et al., 2022].

Prior MoE Transformer Training

Fixed Expert Capacity Constraint

Prior work defines **fixed expert capacity** [Fedus et al., 2022]:

- **Token dropping:** Excess tokens dropped
- **Padding:** Insufficient experts padded
- Static buffer allocation → inefficiency

Expert Capacity Formula

$$\text{Expert Capacity} = \frac{\text{num tokens}}{\text{num experts}} \times \text{capacity factor}$$

- Capacity factor: multiplier on uniform distribution
- Tradeoff: computation vs. quality
- Higher factor → less dropping, more padding

(A) Batched Matrix Multiplication
Compute a set of independent matrix multiplications of the same size in parallel.

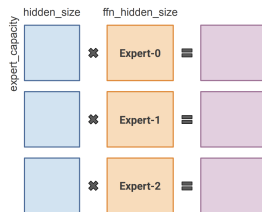


Figure: Expert capacity in Switch Transformer.

Impact of Token Dropping

Token dropping significantly impacts model quality:

- Capacity factor = 1: 0.15 validation loss reduction
- Avoiding token dropping: 0.26 reduction (**1.73× larger**)
- Higher capacity enables better performance than dense models

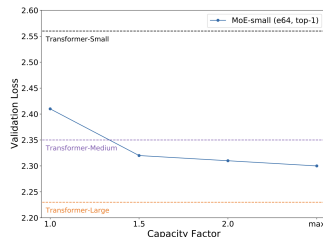
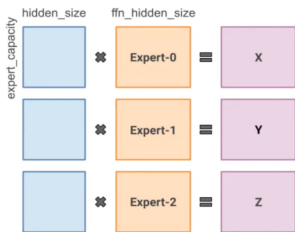


Figure 2. MoEs Trained on The Pile with Different Capacity Factors. The loss reached by the MoE models decreases significantly as expert capacity is increased, but at the cost of additional computation. The lowest loss is achieved by the “max” capacity factor model, which avoids dropping tokens through the dynamic capacity factor mechanism proposed by [Hwang et al. \(2022\)](#).

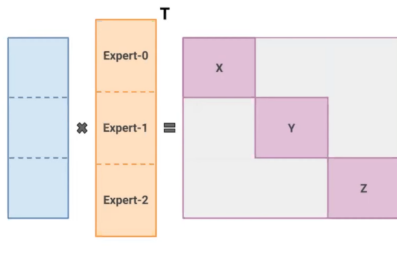
Figure: Effect of capacity factor on model quality.

Idea: Block Diagonal Reformulation (1)

(Current) Batched Matmul
Parallel, but constrained expert computation.

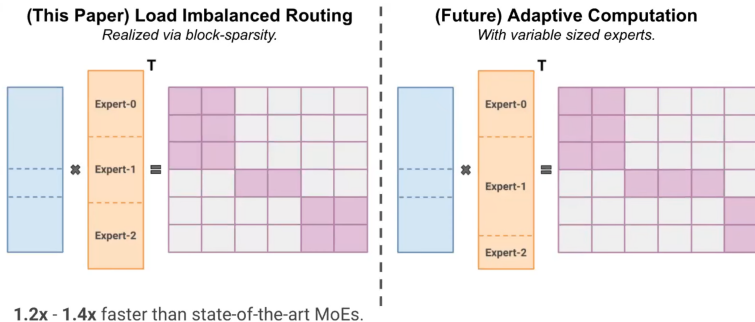


(Reformulation) Block Diagonal Matmul
Expert computation with block diagonal matrices.



MegaBlocks reformulates MoE computation as block-diagonal matrix operations, enabling efficient GPU implementation without token dropping.

Idea: Block Diagonal Reformulation (2)



Handling load-imbalanced routing through block-sparse operations allows dynamic token assignment without capacity constraints.

Recap: Sparse Matrix Product Notation

Notation System

Borrowed from Triton (Tillet et al., 2019) to describe sparse/dense matrix multiplication operations

Three-Character String Format

- Each character is either **S** (Sparse) or **D** (Dense)
- Order: **Output** $\rightarrow$ **Left Input** $\rightarrow$ **Right Input**

Examples

- **SDD**: Sparse output, Dense $\times$ Dense inputs
 - Also known as Sampled Dense-Dense Matrix Multiplication (SDDMM)
- **DSD** vs **DDS**: Different forms of Sparse Matrix-Dense Matrix Multiplication (SpMM)
- **SDD^T**: SDD with transposed right-hand input

dMoE Forward Pass Pseudocode

MegaBlocks dMoE Implementation

```
1  # x.shape: (num_tokens, hidden_size)
2  def dmoe_forward(self, x):
3      # (1) Assign tokens to experts.
4      #
5      # indices.shape: (num_tokens)
6      # weights.shape: (num_tokens)
7      indices, weights = router(x)
8
9      # (2) Create the sparse matrix topology.
10     #
11     # This describes the matrix in Figure of Block Diagonal Reformulation slide.
12     topology = make_topology(indices)
13
14     # (3) Permute the tokens to group by expert.
15     x = padded_gather(x, indices)
16
17     # (4): Compute the expert layers.
18     #
19     # inner_dim = ffn_hidden_size * num_experts
20     # self.w1.shape: (hidden_size, inner_dim)
21     # self.w1.shape: (inner_dim, hidden_size)
22     x = sdd(x, self.w1, topology)
23     x = dsd(x, self.w2)
24
25     # (5) Un-permute the tokens and scale.
26     x = padded_scatter(x, indices)
27     return x * weights
```

MegaBlocks Approach

One Liner Summary

Reformulate MoE computation in terms of **block-sparse operations**

New Block-Sparse GPU Kernels

Efficiently handle the dynamism present in MoEs

Benefits

- Never drops tokens
- Maps efficiently to modern hardware
- Enables significant speedups

- MegaBlocks presents an efficient system for MoE training on GPUs
- Block-sparse operations eliminate the quality-efficiency tradeoff
- Significant performance improvements over existing frameworks
- Enables scalable MoE training without token dropping

Discussion Question

Hardware Trend vs. MoE Direction

MoE models use less computation relative to their model size. However, on modern GPUs, computation is becoming relatively cheap while memory is increasingly expensive.

Key Question

This seems opposite to hardware trends—does this direction still make sense?



Fedus, W., Zoph, B., and Shazeer, N. (2022).

Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.

Journal of Machine Learning Research.

JMLR.



Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017).

Outrageously large neural networks: The sparsely-gated mixture-of-experts layer.

In *International Conference on Learning Representations*.

Thank you!
junormoo@snu.ac.kr